

Implement a proper Easy, Medium, and Hard difficulty system for this Flutter Shikaku game.

Before changing anything, inspect the existing repository, especially:

- `lib/logic/generator.dart`
- `lib/logic/validator.dart`
- `lib/models/puzzle.dart`
- `lib/state/game_controller.dart`
- Existing level/settings UI
- Existing tests

The current generator creates a valid partition and places one clue inside each solution rectangle. It scales rows, columns, and maximum rectangle area by level, but it does not measure logical difficulty or guarantee a unique solution.

Goal

Change puzzle generation to use this pipeline:

1. Generate a candidate partition.
2. Place clues.
3. Enumerate all possible rectangles for every clue.
4. Solve and analyze the puzzle.
5. Reject puzzles without exactly one solution.
6. Reject puzzles that do not match the requested difficulty.
7. Retry with a deterministic sequence of candidate boards.
8. Return the closest valid unique puzzle if the exact target is not found within the attempt limit.

Do not classify difficulty using grid size alone.

1. Add difficulty models

Create `lib/models/puzzle_difficulty.dart`.

Add:

```
enum PuzzleDifficulty {  
  easy,  
  medium,  
  hard,  
}
```

Add an immutable `DifficultyAnalysis` model containing at least:

```

final int solutionCount; // Stop counting at 2.
final int clueCount;
final int cellCount;
final double averageInitialCandidates;
final int maxInitialCandidates;
final int initialForcedClues;
final int propagationRounds;
final int forcedPlacements;
final int guesses;
final int maxSearchDepth;
final int visitedNodes;
final int score;

```

Include:

```

double get initialForcedRatio;
bool get isUnique => solutionCount == 1;

```

Add an immutable `DifficultyProfile` that defines generation settings and acceptance criteria.

Suggested initial profiles:

Easy

- Grid approximately 6×7 to 7×8.
- `maxSearchDepth == 0`.
- `initialForcedRatio >= 0.25`.
- `averageInitialCandidates <= 3.0`.
- Difficulty score from 0 through 35.
- Prefer smaller rectangles.
- Prefer clue positions near the center of their solution rectangle.

Medium

- Grid approximately 7×8 to 8×9.
- `maxSearchDepth <= 1`.
- Difficulty score from 30 through 65.
- Use mostly random clue positions.
- Allow longer propagation chains and more candidate ambiguity.

Hard

- Grid approximately 8×9 to 9×11.
- Difficulty score of at least 55.
- Prefer low initial forced ratios.

- Allow `maxSearchDepth` from 1 through 3.
- Reject pathological puzzles requiring more than a reasonable node limit, such as 5,000 visited nodes.
- Prefer clue positions away from the center when possible, because edge-biased clue positions often create more possible rectangle placements.

Treat these values as centralized tuning constants, not scattered magic numbers.

2. Build a candidate enumerator

Create `lib/logic/puzzle_solver.dart`.

For every clue with value `area`, enumerate every valid rectangle that:

- Has `width * height == area`.
- Contains the clue cell.
- Is fully inside the grid.
- Contains exactly one clue.
- Has positive width and height.

Efficient enumeration:

1. Iterate through factor pairs of the clue value.
2. For each possible width and height, calculate all possible top-left positions that would contain the clue.
3. Reject rectangles outside the board.
4. Reject rectangles containing another clue.
5. Deduplicate rectangles.

Create an internal candidate type containing:

```
final int clueIndex;
final GridRect rect;
final Set<int> cellIndices;
```

Do not use the stored generated solution while solving. The solver must solve using clues only.

3. Implement constraint propagation

The puzzle has two exact-cover-style constraints:

- Every clue must select exactly one rectangle.
- Every board cell must be covered exactly once.

Repeatedly apply forced choices:

Clue single

If an unresolved clue has exactly one remaining compatible candidate, select it.

Cell single

If an uncovered cell can only be covered by one remaining candidate, select that candidate.

When selecting a candidate:

- Mark its clue as resolved.
- Mark its cells as occupied.
- Remove all other candidates belonging to the same clue.
- Remove all candidates that overlap any occupied cell.
- Detect contradictions immediately:
 - An unresolved clue has zero candidates.
 - An uncovered cell has zero possible covering candidates.
 - Two selected rectangles overlap.

Continue until:

- The puzzle is solved.
- A contradiction occurs.
- No forced moves remain.

Track:

- Propagation rounds.
- Forced placements.
- Initial candidate counts.
- Initial forced clue count.

4. Implement bounded search and uniqueness checking

When propagation stops before completion:

1. Look at unresolved clue constraints and uncovered cell constraints.
2. Select the constraint with the fewest remaining candidates greater than one.
3. Recursively try each candidate.
4. Apply propagation after every choice.
5. Track:
 6. Guess count.
 7. Maximum recursion depth.

8. Visited search nodes.
9. Stop immediately after finding two solutions.

Expose an API similar to:

```
DifficultyAnalysis analyze(Puzzle puzzle);

int countSolutions(
    Puzzle puzzle, {
    int limit = 2,
});
```

The analysis should describe the first successful solve path while solution counting must continue until zero, one, or two solutions have been established.

Put safety limits on search depth and visited nodes so malformed candidates cannot freeze the application.

5. Calculate a difficulty score

Calculate a normalized score from 0 through 100.

Use a formula based primarily on logical ambiguity and search complexity, not board size.

Suggested weighting:

```
25% average candidate ambiguity
20% inverse initial forced ratio
15% propagation-chain length
25% maximum search depth
10% visited search nodes
5% board size
```

Normalize every component to a value from 0 through 1 before combining them.

Suggested examples:

```
ambiguity =
    ((averageInitialCandidates - 1) / 5).clamp(0.0, 1.0);

forcedDifficulty =
    (1.0 - initialForcedRatio).clamp(0.0, 1.0);

chainDifficulty =
```

```
(propagationRounds / 10).clamp(0.0, 1.0);

searchDifficulty =
    (maxSearchDepth / 3).clamp(0.0, 1.0);
```

Use a logarithmic or capped normalization for visited nodes so one extreme puzzle does not dominate the entire score.

The profile acceptance rules should use both the score and hard safety constraints. For example, an Easy puzzle must never require search even if its numerical score happens to be low.

6. Refactor the generator

Update the generator API to:

```
Puzzle generate(
    int level, {
        PuzzleDifficulty difficulty = PuzzleDifficulty.medium,
        int? seed,
    });
```

Requirements:

- Preserve deterministic generation.
- The same level, difficulty, and seed must always produce the same puzzle.
- Include the difficulty in the generated seed so Easy level 1 and Hard level 1 are different.
- Generate up to approximately 200 candidate puzzles.
- Analyze every candidate.
- Accept only candidates with exactly one solution.
- Return immediately when a candidate matches the requested profile.
- Track the closest valid unique candidate during retries.
- If no exact candidate is found, return the closest unique candidate rather than crashing or looping forever.
- Never return a puzzle with a 1-cell solution rectangle.
- Never return a puzzle with zero or multiple solutions.

Create private generation parameters per difficulty, including:

- Rows and columns.
- Maximum rectangle area.
- Partition stop probability.
- Clue-position strategy.
- Attempt count.
- Search safety limits.

Use level to create progression inside each difficulty. For example, later Easy levels can grow slightly larger while remaining logically Easy.

7. Improve clue placement

Replace fully random clue placement with a configurable strategy.

Easy

Prefer cells close to the center of the solution rectangle.

Medium

Choose mostly random cells.

Hard

Prefer cells closer to rectangle edges or corners, but still let the analyzer decide whether the resulting puzzle is genuinely Hard.

Add a small amount of randomness so puzzles do not all look mechanically similar.

Difficulty must always be verified by the solver after clue placement.

8. Update the Puzzle model

Add:

```
final PuzzleDifficulty difficulty;
```

Optionally include:

```
final DifficultyAnalysis? difficultyAnalysis;
```

The analysis may be included only in debug builds if keeping it in production is unnecessary.

Keep the existing solution list because hints and the wand use it. However, only accept puzzles where the solver confirms that the solution is unique.

9. Update GameController

Make `GameController` preserve the selected difficulty.

Add APIs similar to:

```
PuzzleDifficulty get difficulty;

void setDifficulty(PuzzleDifficulty difficulty);

void loadLevel(
    int level, {
    PuzzleDifficulty? difficulty,
});
```

Requirements:

- Reset preserves the current difficulty.
- Changing difficulty reloads the current level.
- Loading another level preserves difficulty unless explicitly overridden.
- Existing hint, wand, reset, timer, undo, and validation behavior must keep working.

10. Add a difficulty selector

Inspect the existing visual system and add an Easy / Medium / Hard selector in the most appropriate existing level or settings interface.

Requirements:

- Match the existing light and dark themes.
- Use a compact segmented control or three-option selector.
- Do not redesign unrelated screens.
- Persist the selected difficulty using the project's existing `shared_preferences` settings system.
- Default existing users to Medium.
- Reload the current puzzle when difficulty changes.
- Clearly show the active difficulty during level selection or gameplay without adding excessive UI clutter.

11. Add tests

Expand the logic tests.

Test at least:

Generator validity

For multiple levels in every difficulty:

- The stored solution covers every cell exactly once.
- Rectangles do not overlap.
- Every rectangle contains exactly one clue.
- Every clue value equals its rectangle area.
- No rectangle has area 1.

Uniqueness

For at least 15–20 levels in each difficulty:

```
expect(solver.countSolutions(puzzle, limit: 2), 1);
```

Determinism

Generating the same level and difficulty twice should produce identical:

- Grid dimensions.
- Clues.
- Solution rectangles.
- Difficulty analysis.

Difficulty matching

Every generated puzzle should satisfy its profile's hard constraints.

Examples:

- Easy puzzles have `maxSearchDepth == 0`.
- Medium puzzles have `maxSearchDepth <= 1`.
- Hard puzzles meet the configured Hard score or complexity condition.

Aggregate ordering

Across a deterministic sample, verify that:

```
average Easy score < average Medium score  
average Medium score < average Hard score
```

Do not require every individual Hard puzzle to score higher than every individual Medium puzzle.

Solver fixtures

Create small manually defined puzzles to test:

- Unique solution.
 - Multiple solutions.
 - No solution.
 - Clue-single propagation.
 - Cell-single propagation.
 - Search-required solution.
 - Search cutoff after two solutions.
-

12. Debugging and calibration

In debug mode, log one compact line when generating a puzzle:

```
Level 12 | hard | 9x10 | clues 18 | score 73 |  
avg candidates 4.8 | forced 1/18 | rounds 9 |  
depth 2 | nodes 37 | solutions 1 | attempts 24
```

Do not log during release builds.

Keep all difficulty thresholds in one location so they can be calibrated later using real player completion time, mistakes, hints, and abandonment data.

Definition of done

The implementation is complete when:

- Players can select Easy, Medium, or Hard.
- The selection is persisted.
- All generated puzzles have exactly one solution.
- Easy puzzles require no backtracking.
- Medium puzzles require more propagation or at most shallow search.
- Hard puzzles demonstrate meaningfully higher ambiguity or search depth.
- Generation remains deterministic.
- Generation has bounded retries and bounded search.
- Existing gameplay features still work.
- `flutter analyze` passes.
- `flutter test` passes.

Implement the changes directly. After implementation, summarize:

1. Files created.
2. Files modified.
3. Difficulty metrics used.
4. Any thresholds that may need future calibration.
5. Test results.